

CO5 AT3-CODING CHALLENGE

SUTDENT NAME: P. VISHNU VARDHAN REDDY

REG. NO: 192425212

COURSE CODE: DSA0108

1. A multinational company wants to develop an Employee Performance Analytics System to evaluate employee productivity across multiple departments, where each department has a varying number of employees and performance scores. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage employee data effectively. The program should compute the top performer in each department, identify the overall best employee, and calculate the average performance score per department. Additionally, the solution must handle variable department sizes, edge cases such as no employees, equal scores, or all low-performing employees, and ensure efficient comparisons, optimized time complexity, and modular program design.

Code:

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <limits>
5 using namespace std;
6
7 class Employee {
8 private:
9     string name;
10    int id;
11    double performanceScore;
12
13    static int totalEmployeesCreated; // static member to track total employees
14
15 public:
16    // Default constructor
17    Employee() : name(""), id(0), performanceScore(0.0) {}
18
19    // Parameterized constructor
```

```

20 Employee(string empName, int empId, double score) {
21     name = empName;
22     id = empId;
23     performanceScore = score;
24     totalEmployeesCreated++;
25 }
26
27 // Getter functions (access specifiers demonstrate encapsulation)
28 string getName() const { return name; }
29 int getId() const { return id; }
30 double getScore() const { return performanceScore; }
31
32 // Static function to access static member
33 static int getTotalEmployees() {
34     return totalEmployeesCreated;
35 }
36
37 // Display employee details
38 void display() const {

```

```

39     cout << " Employee ID: " << id
40     << " | Name: " << name
41     << " | Score: " << performanceScore << endl;
42 }
43 };
44
45 // Initialize static member
46 int Employee::totalEmployeesCreated = 0;
47
48 class Department {
49 private:
50     string deptName;
51     vector<Employee> employees;
52
53 public:
54     Department() : deptName("Unnamed") {}
55     Department(string name) : deptName(name) {}
56
57     void addEmployee(const Employee &emp) {

```

```

58 employees.push_back(emp);
59 }
60
61 string getDeptName() const { return deptName; }
62
63 int getEmployeeCount() const {
64     return employees.size();
65 }
66
67 // Compute top performer in this department
68 Employee getTopPerformer() const {
69     if (employees.empty()) {
70         return Employee(); // Edge case: no employees
71     }
72
73     Employee topEmp = employees[0];
74     for (size_t i = 1; i < employees.size(); i++) {
75         if (employees[i].getScore() > topEmp.getScore()) {
76             topEmp = employees[i];
77         }
78     }
79     return topEmp;
80 }
81
82 // Calculate average performance score for department
83 double getAverageScore() const {
84     if (employees.empty()) {
85         return 0.0; // Edge case: no employees, avoid division by zero
86     }
87
88     double total = 0.0;
89     for (const auto &emp : employees) {
90         total += emp.getScore();
91     }
92     return total / employees.size();
93 }
94
95 void displayAllEmployees() const {

```

```

96 if (employees.empty()) {
97     cout << " No employees in this department.\n";
98     return;
99 }
100 for (const auto &emp : employees) {
101     emp.display();
102 }
103 }
104 };
105
106 class Company {
107 private:
108     vector<Department> departments;
109
110 public:
111     void addDepartment(const Department &dept) {
112         departments.push_back(dept);
113     }
114

```

```

115 // Identify overall best employee across all departments
116 void findOverallBestEmployee() const {
117     bool found = false;
118     Employee bestEmp;
119     string bestDept;
120
121     for (const auto &dept : departments) {
122         if (dept.getEmployeeCount() == 0) continue; // skip empty depts
123
124         Employee topInDept = dept.getTopPerformer();
125         if (!found || topInDept.getScore() > bestEmp.getScore()) {
126             bestEmp = topInDept;
127             bestDept = dept.getDeptName();
128             found = true;
129         }
130     }
131
132     cout << "\n=== OVERALL BEST EMPLOYEE ===\n";
133     if (!found) {

```

```

134 cout << "No employees found in the company.\n";
135 } else {
136 cout << "Department: " << bestDept << endl;
137 bestEmp.display();
138 }
139 }
140
141 // Generate full report
142 void generateReport() const {
143 cout << "\n==== EMPLOYEE PERFORMANCE ANALYTICS REPORT =====\n";
144
145 if (departments.empty()) {
146 cout << "No departments available.\n";
147 return;
148 }
149
150 for (const auto &dept : departments) {
151 cout << "\nDepartment: " << dept.getDeptName() << endl;
152 dept.displayAllEmployees();
153
154 cout << " Average Score: ";
155 if (dept.getEmployeeCount() == 0)
156 cout << "N/A (No employees)\n";
157 else
158 cout << dept.getAverageScore() << endl;
159
160 cout << " Top Performer: ";
161 if (dept.getEmployeeCount() == 0) {
162 cout << "N/A (No employees)\n";
163 } else {
164 Employee top = dept.getTopPerformer();
165 cout << top.getName() << " (Score: " << top.getScore() << ")\n";
166 }
167 }
168
169 findOverallBestEmployee();
170
171 cout << "\nTotal Employees Created (static count): "
172 << Employee::getTotalEmployees() << endl;

```

```

173 }
174 };
175
176 int main() {
177     Company company;
178
179     // Department 1: Sales
180     Department sales("Sales");
181     sales.addEmployee(Employee("Alice", 101, 85.5));
182     sales.addEmployee(Employee("Bob", 102, 92.3));
183     sales.addEmployee(Employee("Charlie", 103, 92.3)); // equal score edge case
184     company.addDepartment(sales);
185
186     // Department 2: IT (all low performers)
187     Department it("IT");
188     it.addEmployee(Employee("David", 201, 40.0));
189     it.addEmployee(Employee("Eve", 202, 35.5));
190     company.addDepartment(it);
191
192     // Department 3: HR (no employees - edge case)
193     Department hr("HR");
194     company.addDepartment(hr);
195
196     // Department 4: Finance
197     Department finance("Finance");
198     finance.addEmployee(Employee("Frank", 301, 95.0));
199     company.addDepartment(finance);
200
201     // Generate the full analytics report
202     company.generateReport();
203
204     return 0;
205 }

```

```

#include <iostream>
#include <string>
#include <vector>
#include <limits>
using namespace std;

class Employee {
private:
    string name;
    int id;
    double performanceScore;

```

```

static int totalEmployeesCreated; // static member to track total employees

public:
// Default constructor
Employee() : name(""), id(0), performanceScore(0.0) {}

// Parameterized constructor
Employee(string empName, int empId, double score) {
name = empName;
id = empId;
performanceScore = score;
totalEmployeesCreated++;
}

// Getter functions (access specifiers demonstrate encapsulation)
string getName() const { return name; }
int getId() const { return id; }
double getScore() const { return performanceScore; }

// Static function to access static member
static int getTotalEmployees() {
return totalEmployeesCreated;
}

// Display employee details
void display() const {
cout << " Employee ID: " << id
<< " | Name: " << name
<< " | Score: " << performanceScore << endl;
}
};

// Initialize static member
int Employee::totalEmployeesCreated = 0;

class Department {
private:
string deptName;
vector<Employee> employees;

public:
Department() : deptName("Unnamed") {}
Department(string name) : deptName(name) {}

void addEmployee(const Employee &emp) {
employees.push_back(emp);
}

string getDeptName() const { return deptName; }

int getEmployeeCount() const {
return employees.size();
}
}

```

```

// Compute top performer in this department
Employee getTopPerformer() const {
    if (employees.empty()) {
        return Employee(); // Edge case: no employees
    }

    Employee topEmp = employees[0];
    for (size_t i = 1; i < employees.size(); i++) {
        if (employees[i].getScore() > topEmp.getScore()) {
            topEmp = employees[i];
        }
    }
    return topEmp;
}

// Calculate average performance score for department
double getAverageScore() const {
    if (employees.empty()) {
        return 0.0; // Edge case: no employees, avoid division by zero
    }

    double total = 0.0;
    for (const auto &emp : employees) {
        total += emp.getScore();
    }
    return total / employees.size();
}

void displayAllEmployees() const {
    if (employees.empty()) {
        cout << " No employees in this department.\n";
        return;
    }
    for (const auto &emp : employees) {
        emp.display();
    }
}

};

class Company {
private:
    vector<Department> departments;

public:
    void addDepartment(const Department &dept) {
        departments.push_back(dept);
    }

    // Identify overall best employee across all departments
    void findOverallBestEmployee() const {
        bool found = false;
        Employee bestEmp;
        string bestDept;
    }

```



```

for (const auto &dept : departments) {
    if (dept.getEmployeeCount() == 0) continue; // skip empty depts

    Employee topInDept = dept.getTopPerformer();
    if (!found || topInDept.getScore() > bestEmp.getScore()) {
        bestEmp = topInDept;
        bestDept = dept.getDeptName();
        found = true;
    }
}

cout << "\n=== OVERALL BEST EMPLOYEE ===\n";
if (!found) {
    cout << "No employees found in the company.\n";
} else {
    cout << "Department: " << bestDept << endl;
    bestEmp.display();
}

// Generate full report
void generateReport() const {
    cout << "\n===== EMPLOYEE PERFORMANCE ANALYTICS REPORT =====\n";

    if (departments.empty()) {
        cout << "No departments available.\n";
        return;
    }

    for (const auto &dept : departments) {
        cout << "\nDepartment: " << dept.getDeptName() << endl;
        dept.displayAllEmployees();

        cout << " Average Score: ";
        if (dept.getEmployeeCount() == 0)
            cout << "N/A (No employees)\n";
        else
            cout << dept.getAverageScore() << endl;

        cout << " Top Performer: ";
        if (dept.getEmployeeCount() == 0) {
            cout << "N/A (No employees)\n";
        } else {
            Employee top = dept.getTopPerformer();
            cout << top.getName() << " (Score: " << top.getScore() << ")\n";
        }
    }

    findOverallBestEmployee();

    cout << "\nTotal Employees Created (static count): "
    << Employee::getTotalEmployees() << endl;
}
};

```

```
int main() {
    Company company;

    // Department 1: Sales
    Department sales("Sales");
    sales.addEmployee(Employee("Alice", 101, 85.5));
    sales.addEmployee(Employee("Bob", 102, 92.3));
    sales.addEmployee(Employee("Charlie", 103, 92.3)); // equal score edge case
    company.addDepartment(sales);

    // Department 2: IT (all low performers)
    Department it("IT");
    it.addEmployee(Employee("David", 201, 40.0));
    it.addEmployee(Employee("Eve", 202, 35.5));
    company.addDepartment(it);

    // Department 3: HR (no employees - edge case)
    Department hr("HR");
    company.addDepartment(hr);

    // Department 4: Finance
    Department finance("Finance");
    finance.addEmployee(Employee("Frank", 301, 95.0));
    company.addDepartment(finance);

    // Generate the full analytics report
    company.generateReport();

    return 0;
}
```

Output:

```
===== EMPLOYEE PERFORMANCE ANALYTICS REPORT =====

Department: Sales
Employee ID: 101 | Name: Alice | Score: 85.5
Employee ID: 102 | Name: Bob | Score: 92.3
Employee ID: 103 | Name: Charlie | Score: 92.3
Average Score: 90.0333
Top Performer: Bob (Score: 92.3)
```

Department: IT

Employee ID: 201 | Name: David | Score: 40

Employee ID: 202 | Name: Eve | Score: 35.5

Average Score: 37.75

Top Performer: David (Score: 40)

Department: HR

No employees in this department.

Average Score: N/A (No employees)

Top Performer: N/A (No employees)

Department: Finance

Employee ID: 301 | Name: Frank | Score: 95

Average Score: 95

Top Performer: Frank (Score: 95)

=== OVERALL BEST EMPLOYEE ===

Department: Finance

Employee ID: 301 | Name: Frank | Score: 95

Total Employees Created (static count): 6

2. A logistics company wants to develop a Fleet Management System to monitor multiple vehicles across different routes, where each route has a varying number of vehicles and trip records. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent vehicle data efficiently. The program should compute the highest distance covered vehicle on each route, identify the overall most efficient vehicle, and calculate the average distance per route. Additionally, the solution must handle variable fleet sizes, edge cases such as no vehicles, identical distances, or zero trips, and ensure efficient data processing, modular design, and optimized comparisons.

Code:

PROGRAM EDITOR

C++

Run

Save

INPUT

2
2
3 10 20 5
2 15 15
0

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 class Vehicle {
6 private:
7     int id;
8     int trips;           // number of trip records
9     int bestDistance;    // highest distance in those trips
10    int totalDistance;    // sum of distances
11
12 public:
13     static int totalVehicles; // static member (counts created vehicles)
14
15     Vehicle() : id(-1), trips(0), bestDistance(0), totalDistance(0) {
16         totalVehicles++;
17     }
18
19     Vehicle(int vid) : id(vid), trips(0), bestDistance(0), totalDistance(0) {
20         totalVehicles++;
21     }
22
23     void addTripDistance(int d) {
24         if (d < 0) return;
25         if (trips == 0) {
26             bestDistance = d;
27         } else {
28             if (d > bestDistance) bestDistance = d;
29         }
30         totalDistance += d;
31         trips++;
32     }
33
34     int getBestDistance() const { return bestDistance; }
35     int getTotalDistance() const { return totalDistance; }
36     bool hasTrips() const { return trips > 0; }
37
38     int getId() const { return id; }
```

```

39 };
40
41 int Vehicle::totalVehicles = 0;
42
43 void computeRouteStats(Vehicle vehicles[], int n,
44                        int &routeTopId, int &routeTopDist,
45                        int &routeAvgDist, bool &hasAnyVehicle) {
46
47     hasAnyVehicle = (n > 0);
48     if (!hasAnyVehicle) {
49         routeTopId = -1;
50         routeTopDist = 0;
51         routeAvgDist = 0;
52         return;
53     }
54
55     int sumOfAllVehicleDistances = 0;
56     routeTopDist = -1;
57     routeTopId = -1;
58
59     for (int i = 0; i < n; i++) {
60         int vd = vehicles[i].getTotalDistance();
61         sumOfAllVehicleDistances += vd;
62
63         int best = vehicles[i].getBestDistance();
64         // If identical best distances occur, keep first found (simple + deterministic)
65         if (best > routeTopDist) {
66             routeTopDist = best;
67             routeTopId = vehicles[i].getId();
68         }
69     }
70
71     // Average distance per route (average over vehicles)
72     // If vehicles exist but all have 0 trips => sum is 0 => avg is 0
73     routeAvgDist = sumOfAllVehicleDistances / n;
74 }
75
76 int main() {
77     int R; // number of routes

```

```

78     cin >> R;
79
80     // Max sizes (small demo). Adjust if needed.
81     const int MAX_VEHICLES = 10;
82     const int MAX_ROUTES = 10;
83
84     int overallBestVehicleId = -1;
85     int overallBestDistance = -1;
86     int overallRoutesWithVehicles = 0;
87
88     for (int r = 0; r < R; r++) {
89         int n; // vehicles on this route
90         cin >> n;
91
92         Vehicle vehicles[MAX_VEHICLES];
93
94         // Create vehicles objects (IDs for demo: 100+r*10+i)
95         for (int i = 0; i < n; i++) {
96             vehicles[i] = Vehicle(100 + r * 10 + i);
97
98             int trips;
99             cin >> trips; // number of trip records for this vehicle
100
101             for (int t = 0; t < trips; t++) {
102                 int d;
103                 cin >> d; // distance for each trip
104                 vehicles[i].addTripDistance(d);
105             }
106
107             int routeTopId, routeTopDist, routeAvgDist;
108             bool hasAnyVehicle;
109
110             computeRouteStats(vehicles, n, routeTopId, routeTopDist, routeAvgDist, hasAnyV
111
112             if (hasAnyVehicle) overallRoutesWithVehicles++;
113
114             // Update overall most efficient vehicle using route-top best distance
115             if (hasAnyVehicle) {

```

```

116         if (routeTopDist > overallBestDistance) {
117             overallBestDistance = routeTopDist;
118             overallBestVehicleId = routeTopId;
119         }
120     }
121
122     // Output per route
123     cout << "Route " << r + 1 << ": ";
124     if (!hasAnyVehicle) {
125         cout << "No vehicles, avg distance = 0\n";
126     } else {
127         cout << "Top vehicle ID = " << routeTopId
128             << ", Top distance = " << routeTopDist
129             << ", Avg distance = " << routeAvgDist << "\n";
130     }
131 }
132
133 cout << "Overall most efficient vehicle: ";
134 if (overallBestVehicleId == -1) {
135     cout << "No vehicles in any route\n";
136 } else {
137     cout << "Vehicle ID = " << overallBestVehicleId
138         << ", Best distance = " << overallBestDistance << "\n";
139 }
140
141 // Show static member usage
142 cout << "Total Vehicle objects created = " << Vehicle::totalVehicles << "\n";
143
144 return 0;

```

```

#include <iostream>
#include <string>
using namespace std;

```

```

class Vehicle {
private:
    int id;
    int trips; // number of trip records
    int bestDistance; // highest distance in those trips
    int totalDistance; // sum of distances

public:
    static int totalVehicles; // static member (counts created vehicles)

    Vehicle() : id(-1), trips(0), bestDistance(0), totalDistance(0) {
        totalVehicles++;
    }
}

```

```

Vehicle(int vid) : id(vid), trips(0), bestDistance(0), totalDistance(0) {
    totalVehicles++;
}

void addTripDistance(int d) {
    if (d < 0) return;
    if (trips == 0) {
        bestDistance = d;
    } else {
        if (d > bestDistance) bestDistance = d;
    }
    totalDistance += d;
    trips++;
}

int getBestDistance() const { return bestDistance; }
int getTotalDistance() const { return totalDistance; }
bool hasTrips() const { return trips > 0; }

int getId() const { return id; }
};

int Vehicle::totalVehicles = 0;

void computeRouteStats(Vehicle vehicles[], int n,
    int &routeTopId, int &routeTopDist,
    int &routeAvgDist, bool &hasAnyVehicle) {

    hasAnyVehicle = (n > 0);
    if (!hasAnyVehicle) {
        routeTopId = -1;
        routeTopDist = 0;
        routeAvgDist = 0;
        return;
    }

    int sumOfAllVehicleDistances = 0;
    routeTopDist = -1;
    routeTopId = -1;

    for (int i = 0; i < n; i++) {
        int vd = vehicles[i].getTotalDistance();
        sumOfAllVehicleDistances += vd;

        int best = vehicles[i].getBestDistance();
        // If identical best distances occur, keep first found (simple +
        deterministic)
        if (best > routeTopDist) {
            routeTopDist = best;
            routeTopId = vehicles[i].getId();
        }
    }

    // Average distance per route (average over vehicles)

```



```

// If vehicles exist but all have 0 trips => sum is 0 => avg is 0
routeAvgDist = sumOfAllVehicleDistances / n;
}

int main() {
int R; // number of routes
cin >> R;

// Max sizes (small demo). Adjust if needed.
const int MAX_VEHICLES = 10;
const int MAX_ROUTES = 10;

int overallBestVehicleId = -1;
int overallBestDistance = -1;
int overallRoutesWithVehicles = 0;

for (int r = 0; r < R; r++) {
int n; // vehicles on this route
cin >> n;

Vehicle vehicles[MAX_VEHICLES];

// Create vehicles objects (IDs for demo: 100+r*10+i)
for (int i = 0; i < n; i++) {
vehicles[i] = Vehicle(100 + r * 10 + i);
int trips;
cin >> trips; // number of trip records for this vehicle

for (int t = 0; t < trips; t++) {
int d;
cin >> d; // distance for each trip
vehicles[i].addTripDistance(d);
}
}

int routeTopId, routeTopDist, routeAvgDist;
bool hasAnyVehicle;

computeRouteStats(vehicles, n, routeTopId, routeTopDist, routeAvgDist,
hasAnyVehicle);

if (hasAnyVehicle) overallRoutesWithVehicles++;

// Update overall most efficient vehicle using route-top best distance
if (hasAnyVehicle) {
if (routeTopDist > overallBestDistance) {
overallBestDistance = routeTopDist;
overallBestVehicleId = routeTopId;
}
}

// Output per route
cout << "Route " << r + 1 << ": ";
if (!hasAnyVehicle) {

```

```

cout << "No vehicles, avg distance = 0\n";
} else {
cout << "Top vehicle ID = " << routeTopId
<< ", Top distance = " << routeTopDist
<< ", Avg distance = " << routeAvgDist << "\n";
}
}

cout << "Overall most efficient vehicle: ";
if (overallBestVehicleId == -1) {
cout << "No vehicles in any route\n";
} else {
cout << "Vehicle ID = " << overallBestVehicleId
<< ", Best distance = " << overallBestDistance << "\n";
}

// Show static member usage
cout << "Total Vehicle objects created = " << Vehicle::totalVehicles << "\n";

return 0;
}

```

Output:

```

Route 1: Top vehicle ID = 100, Top distance = 20, Avg distance = 32
Route 2: No vehicles, avg distance = 0
Overall most efficient vehicle: Vehicle ID = 100, Best distance = 20
Total Vehicle objects created = 22

```